Exceptions

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **debug** a crash from its stack trace.
2. Be able to **identify** the exception a snippet throws.
3. Be able to **classify** an exception as checked or unchecked.
4. Be able to **explain** why exceptions beat `if` guards.
5. Be able to **trace** execution into and out of a `catch` block.

Review Book System

Review last week's `Book` !

You have met these already

```
Exception in thread "main" java.lang.NullPointerException
```

Every time your program died on a `null` reference or a bad index, Java built an **`object`**, handed it to the JVM, and nobody caught it.

This week we stop treating that as the end of the story.

An exception is an object

```
new IllegalArgumentException("loan days cannot be negative");
```

Same `new`, same constructor, same reference as any `Book` – you were building these last week.

What makes it special is what you can do with it: `throw` it, and it interrupts everything.

It carries what it knows

- Its **type** – `NullPointerException`, `ArithmeticException`, …
- Its **message** – whatever string the constructor got.
- Its **stack trace** – every frame that was open when it happened.

The type says *what*. The trace says *where*. Together, that is most of a debugging session.

Read the stack trace

Three frames deep, one bad assumption...

→ Step through it in Java Visualizer.

What does the trace say?

In JavaVisualizer, just the exception:

`java.lang.NullPointerException`

In Eclipse, the exception and the stack trace!

```
Exception in thread "main" java.lang.NullPointerException:
  Cannot invoke "String.toUpperCase()" because "ogTitle" is null
        at Main.titleOf(Main.java:5)
        at Main.printSlot(Main.java:9)
        at Main.main(Main.java:17)
```

Read it **top down**!

What does the trace say?

```
Exception in thread "main" java.lang.NullPointerException:
  Cannot invoke "String.toUpperCase()" because "ogTitle" is null
        at Main.titleOf(Main.java:5) <-- The CRASH is here!
        at Main.printSlot(Main.java:9)
        at Main.main(Main.java:17)
```

Java even names the reference that was `null`: `ogTtitle`. The top frame is where it blew up.

What does the trace say?

```
Exception in thread "main" java.lang.NullPointerException:
  Cannot invoke "String.toUpperCase()" because "ogTitle" is null
        at Main.titleOf(Main.java:5)
        at Main.printSlot(Main.java:9)
        at Main.main(Main.java:17) <-- The BUG is here!
```

But line 17 chose the bad slot. The top frame is where it *died*; somewhere below is where it *went wrong*.

Tophat Q1

A quick check on stack traces.

→ Open Tophat to vote.

Which one throws?

```
String[] shelf = {"Dune", "Emma"};

System.out.println(shelf[5]);
System.out.println(shelf[0].charAt(99));
System.out.println(Integer.parseInt("14 days"));
System.out.println(7 / 0);
```

Four lines, four *different* exceptions.

Check it in Java Visualizer

The Throwable family

```
Throwable
├── Error                       // the JVM is in trouble; not yours to fix
└── Exception
    ├── RuntimeException        // UNCHECKED
    │   ├── NullPointerException
    │   ├── ArrayIndexOutOfBoundsException
    │   └── IllegalArgumentException
    ├── IOException             // CHECKED
    ├── InterruptedException    // CHECKED
    └── ...                     // CHECKED
```

Exceptions we care about – Errors are bigger than us.

Checked or unchecked?

The rule is a *position in that tree*, nothing else:

- Is it a `RuntimeException`? → **unchecked**. Handle it if you like.
- Otherwise → **checked**. You MUST deal with it. The compiler says so! :)

Importantly: Children are a type of their parent, i.e. a `NullPointerException` is a type of `RuntimeException` and is therefore unchecked.

The compiler is the tell

```
Scanner in = new Scanner(new File("catalog.txt"));
```

Will not compile. `FileNotFoundException` is checked, and we have not said what happens when the file is missing.

Wednesday is about answering that question.

Tophat Q2

A quick check on checked versus unchecked.

→ Open Tophat to vote.

Could we just check first?

```
public static String titleOf(String[] shelf, int slot) {
    if (slot < 0 || slot >= shelf.length) return null;
    if (shelf[slot] == null) return null;
    return shelf[slot].toUpperCase();   // never crashes now!
}
```

Run it in Java Visualizer

Could we just check first?

```
titleOf(shelf, 1);   // null -- the slot is empty
titleOf(shelf, 9);   // null -- there IS no slot 9
```

Two different mistakes. One answer.

Could we just check first?

```
String title = titleOf(shelf, 9);
System.out.println(title.length());  // NullPointerException!
```

The guard did not remove the crash. It moved it away from the mistake.

An exception is how a method says *"I cannot do this, and here is exactly why"* – without inventing a fake return value.

You already called one

```java
// from last week's Book.java
public void passDay() {
  if (loanDays == 0) {
    throw new IllegalStateException("'" + title + "' is due!");
  }
  loanDays--;
}
```

`Book` was throwing at you last Friday.

2 Ways to Handle

You can guard against it...

```
if (book.getLoanDaysRemaining() > 0) {
  book.passDay();
}
```

... or catch it! Either work. It's your design decision.

```
try {
  book.passDay();
} catch (IllegalStateException e) {
  // Handle `e` here!
}
```

The exception is an object, remember

```
} catch (IllegalStateException e) {
  System.out.println(e.getMessage());   // 'Dune' is due!
}
```

`e` is a reference like any other. Catching it is how you get your hands on the thing those stack traces were describing.

Tophat Q3

A quick check on exception handling.

→ Open Tophat to vote.

Questions?